

PRESIDENCY UNIVERSITY

CSE324: Database Management Systems Laboratory

Class 01: Introduction to Database Systems & Relational Model

From Raw Data to an Actual Relational Database

Afzal Hossan
Lecturer
Department of Computer Science and Engineering
Presidency University

Today's Learning Path



You should understand

Database/DBMS, relations, attributes, tuples, domains, keys and integrity.

You should be able to do

Create, populate and inspect a small relational database in MySQL Workbench.

Part I

Why do we need a database?

Opening Problem: One Giant Spreadsheet?

Suppose a university stores everything in one sheet:

Student ID	Student Name	Department	Dept. Office	Course
231001	Rahim	CSE	Room 501	CSE324
231002	Karim	CSE	Room 501	CSE323
231003	Nabila	EEE	Room 402	EEE301
231001	Rahim	CSE	Room 501	CSE323

Likely problems

Repeated data, inconsistent copies, difficult updates and accidental loss.

Database idea

Store related data systematically and let a DBMS enforce rules and relationships.

Data, Information, Database, DBMS

Data vs Information

Data: raw facts such as 231001, `Rahim`, `CSE`.

Information: organized data with meaning, e.g., "Student 231001 is Rahim from CSE."

Database

An organized collection of **related data**.

DBMS

Software that defines, stores, retrieves, modifies and controls database data.

Examples: MySQL, PostgreSQL, SQL Server, Oracle.

Database system

Database + DBMS + users + applications + hardware + procedures.

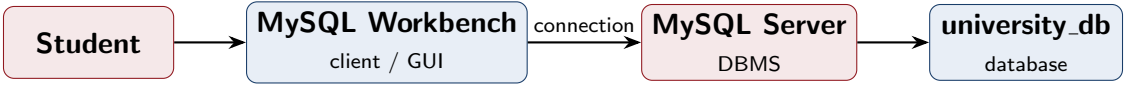
Why Use a DBMS Instead of Separate Files?

Separate files	DBMS-managed data
Duplicate copies are easy to create	Redundancy can be controlled
Updates may become inconsistent	Integrity rules improve consistency
Relationships are handled manually	Keys can represent relationships
Security is scattered across files/apps	Centralized authorization is possible
Sharing and concurrent access are difficult	Multiple users can be supported
Recovery is difficult to coordinate	DBMS facilities support recovery

Important

A DBMS does not magically remove all redundancy. It helps us **control unnecessary redundancy** and protect data quality.

Our Laboratory Environment



Connection information

Hostname, port, username and password tell Workbench how to reach the server.

Metadata

Data about structure: table names, columns, data types, keys and constraints.

Part II

The relational model

The Relational Model: One Table, Four Core Ideas

StudentID	StudentName	DeptID
231001	Rahim	CSE
231002	Nabila	EEE
231003	Karim	CSE

- **Relation** $\approx$ table
- **Tuple** $\approx$ row/record
- **Attribute** $\approx$ column
- **Domain** $=$ permitted kind/set of values

Relation schema: `Student(StudentID, StudentName, DeptID)`

Domain, Data Type, and NULL

Domain

The allowed kind/set of values for an attribute.

- StudentName → character strings
- DateOfBirth → valid dates
- Credits → small integers

Common SQL data types

`INT`, `CHAR(n)`, `VARCHAR(n)`, `DATE`, `DECIMAL(p,s)`, `BOOLEAN`

NULL

`NULL` means a value is absent/unknown/not provided. It is not the same as 0 or an empty string.

Schema, Instance, Degree, and Cardinality

Schema

The relation's structure.

`Student(StudentID, StudentName, DeptID)`

Instance / state

The actual tuples stored at a particular moment.

Degree

Number of attributes.

For the example Student relation: **3**.

Cardinality

Number of tuples currently stored.

For three student rows: **3**.

Basic Properties of a Relation

- a relation has a unique name within its schema;
- attributes have names and domains;
- each tuple represents one record;
- duplicate tuples are not part of the mathematical relational model;
- tuple order has no logical meaning;
- attribute order does not define meaning;
- each attribute value should represent one atomic value in the basic model;
- relationships are represented through matching key values.

Do not confuse implementation with theory

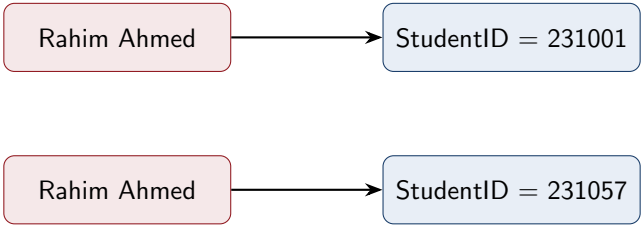
A displayed SQL result may appear ordered, but a relation itself has no inherent row order unless a query explicitly requests one.

Part III

Keys and integrity

Why Do We Need Keys?

Consider two students named "Rahim Ahmed." A name alone may not uniquely identify a person.



A key gives a tuple a reliable identity.

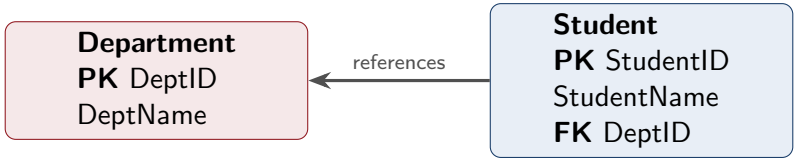
The Key Family

Super key	Any attribute set that uniquely identifies a tuple.
Candidate key	A **minimal** super key; no unnecessary attribute remains.
Primary key	The candidate key selected as the main identifier.
Alternate key	A candidate key not chosen as the primary key.
Composite key	A key formed from two or more attributes.
Foreign key	Attribute(s) that reference a candidate/primary key in a related relation.

Example

If both `StudentID` and `Email` are unique, each can be a candidate key. If `StudentID` is chosen as the primary key, `Email` becomes an alternate key.

Primary Key, Foreign Key, Parent and Child



Parent / referenced relation

`Department` owns the referenced key value.

Child / referencing relation

`Student` stores `DeptID` as a foreign key.

Entity Integrity and Referential Integrity

Entity integrity

The primary key must identify each tuple.

- primary-key values are unique;
- a primary-key value cannot be `NULL`.

Referential integrity

A foreign-key value must reference an existing parent key value, unless the design permits `NULL`.

- `CSE` exists $\rightarrow$ valid;
- `LAW` absent $\rightarrow$ reject.

Keys identify records; integrity rules protect those identities and relationships.

Part IV

Build the first relational database

MySQL Workbench: What You Need Today

- open a saved/local MySQL connection;
- use the SQL Editor;
- execute the selected/current statement;
- read the Result Grid and Action Output;
- inspect the Schemas panel;
- refresh when a new database/table is not visible;
- save the script as `L01_university.sql`.

Focus

Today SQL is a **tool for observing database concepts**. Formal DDL, DML and querying syntax is taught later.

Build 1: Database and Department Relation

```sql
CREATE DATABASE university_db;
USE university_db;

CREATE TABLE Department (
    DeptID CHAR(3) PRIMARY KEY,
    DeptName VARCHAR(100) NOT NULL
);

INSERT INTO Department (DeptID, DeptName)
VALUES
('CSE', 'Computer Science and Engineering'),
('EEE', 'Electrical and Electronic Engineering'),
('BBA', 'Business Administration');

SELECT * FROM Department;
```

Build 2: Student Relation

```
CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Email VARCHAR(100),
    DateOfBirth DATE,
    DeptID CHAR(3) NOT NULL,
    FOREIGN KEY (DeptID)
        REFERENCES Department(DeptID)
);
```

Read the design

Degree = 5; primary key = `StudentID`.

Read the relationship

`DeptID` is a foreign key referencing `Department(DeptID)`.

Build 3: Insert Students and Inspect the Relation

```
INSERT INTO Student
(StudentID, StudentName, Email, DateOfBirth, DeptID)
VALUES
('231001', 'Rahim Ahmed',
 'rahim@pu.edu.bd', '2004-05-10', 'CSE'),
('231002', 'Nabila Islam',
 'nabila@pu.edu.bd', '2003-11-20', 'EEE'),
('231003', 'Karim Hasan',
 'karim@pu.edu.bd', '2004-01-15', 'CSE');

SELECT * FROM Student;
```

After insertion: degree = 5, cardinality = 3.

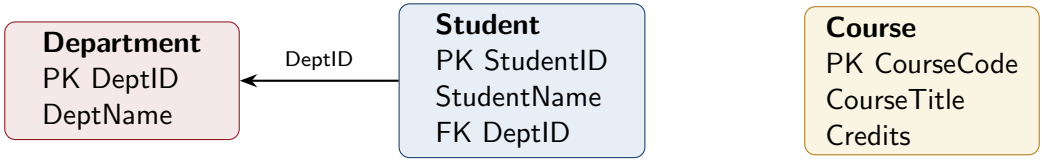
Build 4: Course Relation

```sql
CREATE TABLE Course (
    CourseCode VARCHAR(10) PRIMARY KEY,
    CourseTitle VARCHAR(100) NOT NULL,
    Credits TINYINT NOT NULL
);

INSERT INTO Course (CourseCode, CourseTitle, Credits)
VALUES
('CSE323', 'Database Management Systems', 3),
('CSE324', 'Database Management Systems Laboratory', 1),
('CSE211', 'Data Structures and Algorithms', 3);

SELECT * FROM Course;
```

What We Have Built



Notice

`Course` is not yet connected to `Student`. That future relationship will require additional structure; we deliberately do not solve it yet.

Part V

Make the DBMS reject bad data

Integrity Experiment: Duplicate Key vs Invalid Reference

Duplicate primary key

```
INSERT INTO Student
(StudentID, StudentName,
 Email, DateOfBirth, DeptID)
VALUES
('231001', 'Another Student',
 'another@pu.edu.bd',
 '2004-02-10', 'CSE');
```

✗ Reject: **entity integrity**

Invalid foreign key

```
INSERT INTO Student
(StudentID, StudentName,
 Email, DateOfBirth, DeptID)
VALUES
('231004', 'Test Student',
 'test@pu.edu.bd',
 '2004-03-10', 'LAW');
```

✗ Reject: **referential integrity**

Integrity Experiment: Missing Required Value

```
INSERT INTO Student
(StudentID, StudentName, Email, DateOfBirth, DeptID)
VALUES
('231005', NULL,
 'missing@pu.edu.bd', '2004-04-10', 'CSE');
```

Expected

✗ Rejected because `StudentName` was declared `NOT NULL`.

What to learn

Do not memorize an error number. Explain **which rule was broken and why the DBMS rejected the row**.

Three Failures, Three Protected Rules

Invalid operation	What is protected	Rule
Duplicate `StudentID`	Unique tuple identity	Entity integrity / PK uniqueness
Nonexistent `DeptID`	Valid parent–child reference	Referential integrity
`NULL` student name	Required attribute value	NOT NULL constraint

Part VI

Challenge, submission, and recap

Design Challenge: What Should Become Separate Relations?

StudentID	StudentName	DepartmentName	CourseCode	CourseTitle
231001	Rahim	CSE	CSE324	DBMS Laboratory
231001	Rahim	CSE	CSE323	DBMS
231002	Nabila	CSE	CSE324	DBMS Laboratory

Identify the things

Student, Department, Course.

Boundary

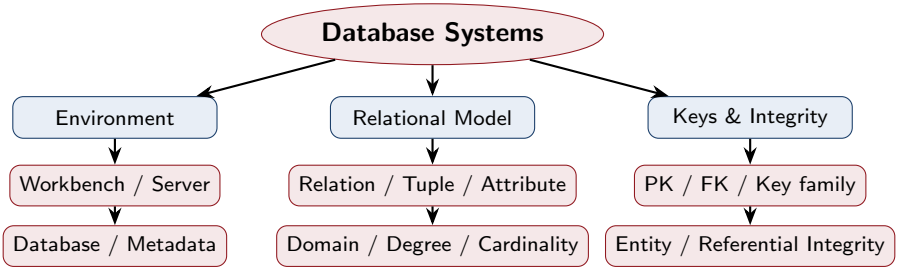
We are recognizing separate relations only. Formal ER modeling and normalization come later.

Lecture 1 Deliverable

`L01_university.sql`

- create/select the database;
- create Department, Student, Course;
- define primary and foreign keys;
- insert valid sample data;
- display table contents;
- keep the three error tests commented;
- add clear SQL comments;
- ensure the main script runs cleanly.

Lecture 1 Concept Map



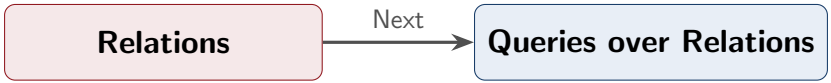
Exit Check

Before you leave, be able to answer:

1. Database vs DBMS: what is the difference?
2. What are a relation, tuple and attribute?
3. Degree vs cardinality: what changes each one?
4. Primary key vs foreign key: what does each do?
5. Why would duplicate StudentID, `DeptID='LAW'`, and a missing required name be rejected?

Next Class: Relational Algebra & Relational Calculus

Today: **What is a relation and how is it protected?**



Next: **How can we describe exactly which information we want from those relations?**

Thank You

Questions?

CSE324 — Database Management Systems Laboratory

Presidency University